

Road Surface Classification

A mini project report

Submitted By,
Bhagawath Narayanan N (241501034)
Swetha Veeramani (241501261)
Harishma R G (241501067)

AI23531 - DEEP LEARNING

in

Department of Artificial Intelligence and Machine Learning Rajalakshmi Engineering College

(Autonomous)

Regulations 2023

BONAFIDE CERTIFICATE

Certified that this the bonafide record of work carried out by Bhagawath Narayanan N (241501034), Sweetha Veeramani (241501261), Harishma R G (241501067) in the mini project titled “Road Surface Classification” as part of the Deep Learning course during the academic year 2026-2027.

TABLE OF CONTENTS

S. No.	Contents	Page No.
	Abstract	4
1	**CHAPTER 1: INTRODUCTION**	5
1.1	Problem Statement and Explanation	5
1.2	Literature Review	5
1.3	Existing System	6
1.4	Proposed System	6
2	**CHAPTER 2: DATA COLLECTION AND PREPROCESSING**	7
2.1	Data Collection – About the Dataset	7
2.2	Architecture Diagram and Workflow Explanation	7
2.3	Data Preprocessing	9
3	**CHAPTER 3: MODEL PERFORMANCE**	10
3.1	Exploratory Data Analysis (EDA)	10
3.2	Model Performance Results	10
3.3	Summary	11
4	**CHAPTER 4: RESULTS AND DISCUSSION**	12
4.1	Overall Performance Comparison	12
4.2	Statistical Significance and Repeated Runs	12
4.3	Model Complexity Analysis	13
4.4	Misclassification and Robustness Analysis	13
4.5	Prediction Demonstration Analysis	13
4.6	Discussion	14
4.7	Conclusion	14
	References	15

ROAD SURFACE CLASSIFICATION USING CONVOLUTIONAL NEURAL NETWORK (CNN)

ABSTRACT

Road surface damage is a significant issue that affects transportation safety, vehicle performance, and road maintenance. Traditional road inspection methods generally require manual observation, which can be time-consuming, expensive, and inconsistent. With the development of artificial intelligence and computer vision, image-based road surface classification provides an efficient approach for identifying damaged road conditions.

This project presents a deep learning-based Road Surface Classification System using a Convolutional Neural Network (CNN). The primary objective is to classify road images into two categories: Broken Road and Non-Broken Road. The dataset consists of road surface images collected from a prepared image dataset and organized into class-specific folders. The images are resized to a fixed dimension, normalized, and divided into training, validation, and testing sets.

A CNN model is developed using TensorFlow and Keras in Google Colab. The model extracts relevant visual features from road images through convolutional and pooling layers. The extracted features are processed by fully connected layers to generate the final classification. The system is evaluated using accuracy, precision, recall, F1-score, and a confusion matrix. Training and validation graphs are also used to analyze model performance.

The proposed system aims to demonstrate how deep learning can support automated road condition assessment. It can serve as a foundation for future applications such as road maintenance planning, intelligent transportation systems, and automated road inspection.

Keywords: Road Surface Classification, Convolutional Neural Network, Deep Learning, Image Processing, TensorFlow, Google Colab.

CHAPTER 1: INTRODUCTION

1.1 Problem Statement and Explanation

Road damage is a common problem in transportation infrastructure. Broken surfaces, potholes, cracks, and uneven road conditions can affect road safety and increase vehicle maintenance costs. Regular road inspection is necessary to identify damaged areas and arrange maintenance activities.

Conventional road inspection depends largely on manual observation and reporting. This process requires time, human effort, and repeated inspection. The results may also vary depending on the inspector and environmental conditions.

The problem addressed in this project is the automatic classification of road surface images into damaged and undamaged categories. Manual inspection is replaced by an image classification approach using deep learning.

The project develops a Convolutional Neural Network that learns visual patterns from labeled road images. Given a road image, the trained model predicts whether the image belongs to the Broken Road or Non-Broken Road category.

The main objectives are:

- To collect and organize road surface images into two classes.
- To preprocess images for effective model training.
- To develop a CNN model for binary image classification.
- To evaluate the model using standard classification metrics.
- To demonstrate predictions on previously unseen road images.
- To establish a foundation for future automated road inspection systems.

1.2 Literature Review

Research in computer vision has demonstrated the usefulness of deep learning for identifying patterns in images. CNNs are particularly suitable for image classification because they can learn spatial features such as edges, textures, shapes, and surface patterns.

Traditional image processing approaches use manually designed features and classification rules. These methods may work under controlled conditions but can struggle when road appearance changes because of lighting, shadows, weather, and surface variation.

Deep learning approaches improve feature extraction by learning representations directly from training images. CNN architectures have been widely studied for visual recognition tasks, including infrastructure inspection and road damage analysis.

Researchers have also explored transfer learning, object detection, and semantic segmentation for more detailed road damage assessment. Transfer learning reuses features learned by pretrained models, while object detection and segmentation can identify the location of individual defects.

The present project implements a basic CNN for binary road surface classification. It focuses on understanding the complete image classification workflow, from dataset preparation to model evaluation. More advanced architectures can be investigated in future work.

1.3 Existing System

In the existing approach, road condition inspection is commonly performed through manual surveys, photographs, and periodic field observations. Inspectors examine road surfaces and record visible damage for maintenance purposes.

Limitations of the Existing System

- Manual inspection requires considerable human effort.
- Large road networks are difficult to inspect frequently.
- Inspection results may vary between individuals.
- Continuous monitoring can be expensive.
- Visual assessment may be affected by lighting and environmental conditions.
- Manual inspection records may take time to process and organize.

Computer-based systems can assist with inspection, but conventional image processing techniques often depend on predefined features and thresholds.

1.4 Proposed System

The proposed system uses a Convolutional Neural Network to classify road surface images automatically. The model is implemented in Python using TensorFlow and Keras in Google Colab.

The dataset is arranged into two classes: Broken Road and Non-Broken Road. Images are resized to 128 × 128 pixels and normalized before being processed by the CNN. The dataset is divided into training, validation, and testing subsets.

The CNN contains convolutional layers for feature extraction, max-pooling layers for spatial reduction, a global average pooling layer, a dense layer, a dropout layer, and a sigmoid output layer.

During training, the model learns to distinguish between the two classes. After training, it is evaluated on a separate test set. The system can also accept a new road image and display its predicted class and model confidence.

Advantages of the Proposed System

- Automates basic road surface image classification.
- Reduces the need for repeated manual image labeling after training.
- Uses open-source tools and a cloud-based development environment.
- Provides measurable performance through classification metrics.
- Can be extended to larger datasets and more advanced models.

The proposed system is intended as a prototype for image-based road condition classification rather than a replacement for professional road inspection.

CHAPTER 2: DATA COLLECTION AND PREPROCESSING

2.1 Data Collection – About the Dataset

The dataset used in this project contains road surface images representing two different road conditions. The images are organized into separate folders according to their class labels.

The two categories are:

1. **Broken Road:** Images containing visible road damage, such as potholes, cracks, or damaged surface areas, depending on the dataset labeling criteria.
2. **Non-Broken Road:** Images representing road surfaces labeled as undamaged or non-broken.

The dataset is stored in a ZIP file and uploaded to the Google Colab environment. After extraction, the image folders are identified automatically, and the number of images in each class is examined.



Figure 2.1: Sample Images from the Road Surface Dataset

Dataset Characteristics

Parameter	Description
Dataset type	Road surface image dataset
Input format	JPG, JPEG, PNG, BMP, or WEBP
Number of classes	2
Class labels	Broken Road and Non-Broken Road
Image dimensions	128 × 128 pixels
Color channels	RGB
Task	Binary image classification

Development platform Google Colab

Programming language Python

Deep learning framework TensorFlow and Keras

The actual number of images in each class must be obtained from the dataset-loading output in Google Colab. The dataset is divided into training, validation, and testing subsets using a 70:15:15 ratio.

Source Code

```
import os
for cls in class_names:
    folder = os.path.join(data_dir, cls)
    count = sum(
        f.lower().endswith(
            (".jpg", ".jpeg", ".png", ".bmp", ".webp")
        )
        for f in os.listdir(folder)
        if os.path.isfile(os.path.join(folder, f))
    )
    print(f"{cls}: {count} images")
print("Total classes:", len(class_names))
```

Expected Output

Broken: [actual count]

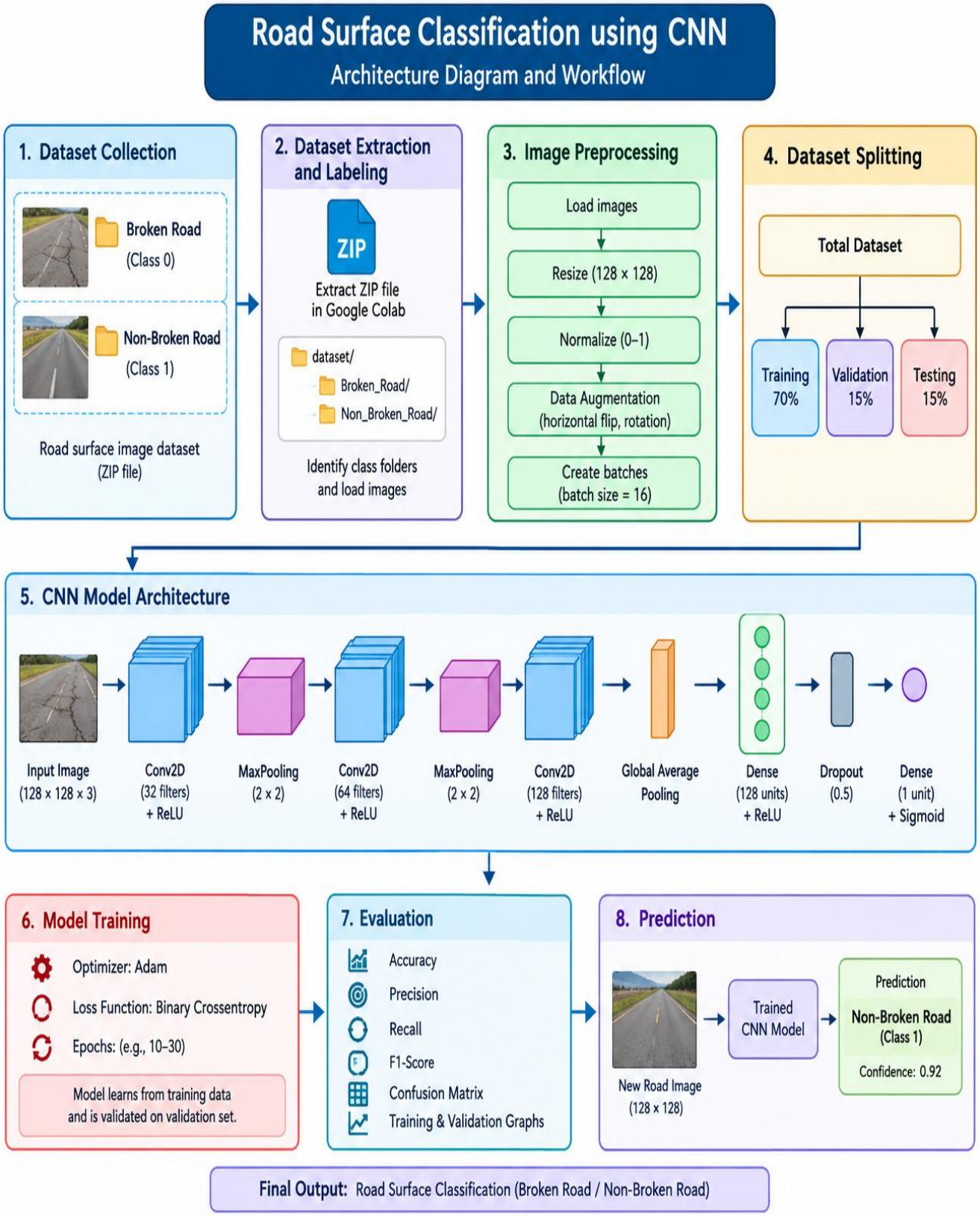
Non-Broken: [actual count]

Total classes: 2

2.2 Architecture Diagram and Workflow Explanation

The proposed system follows a sequence of operations, beginning with dataset collection and ending with road surface prediction.

Architecture Diagram



Road Surface Classification System using CNN

Dataset of Road Images

↓

Dataset Extraction and Class Labeling

↓

Image Preprocessing and Resizing

↓

Dataset Splitting

↓

CNN Feature Extraction

↓

Max-Pooling Layers

↓

Feature Aggregation and Dense Layer

↓

Sigmoid Classification Output

↓

Broken Road / Non-Broken Road Prediction

↓

Model Evaluation and Result Visualization

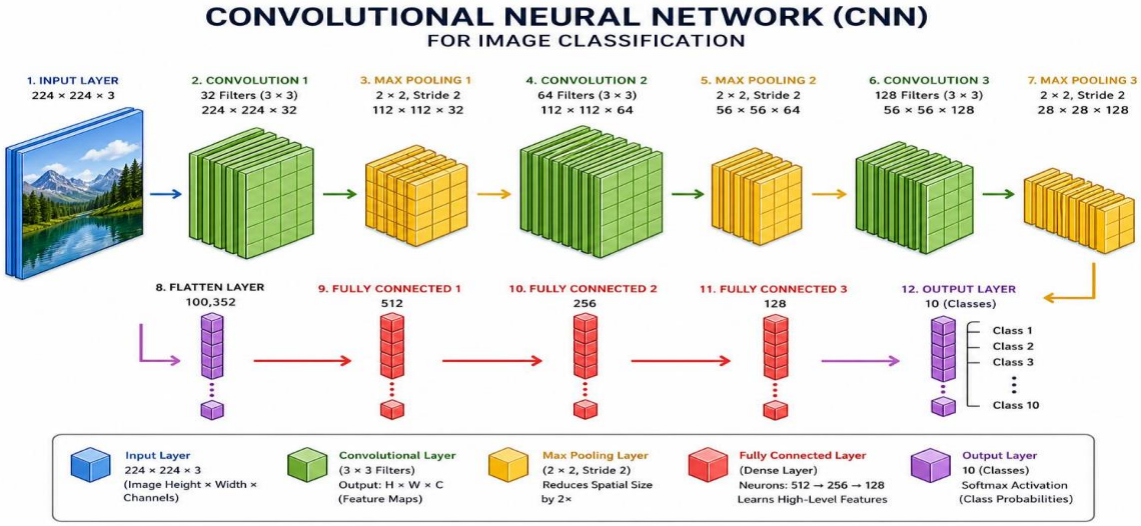


Figure 2.2: Architecture of the Proposed CNN Model

Workflow Explanation

1. Dataset Collection: Road surface images are collected and organized into class-specific folders.

2. Dataset Extraction: The ZIP file is extracted in Google Colab, and the class folders are identified.

3. Image Preprocessing: Images are resized to a standard dimension of 128 × 128 pixels. Pixel values are normalized to a range between 0 and 1.

4. Dataset Splitting: The dataset is divided into training, validation, and testing sets. The training set teaches the model, the validation set helps monitor learning, and the test set evaluates performance on held-out images.

5. Feature Extraction: Convolutional layers identify image patterns such as edges, textures, cracks, and surface variations.

6. Pooling: Max-pooling layers reduce spatial dimensions while retaining important features.

7. Classification: Global average pooling and dense layers combine the extracted features. The sigmoid output produces a probability for class index 1.

8. Prediction: A probability below 0.5 indicates class index 0, while a probability of 0.5 or above indicates class index 1.

9. Evaluation: The trained model is evaluated using test accuracy, precision, recall, F1-score, and a confusion matrix.

2.3 Data Preprocessing

Data preprocessing is an important step in preparing image data for deep learning. Images may have different sizes, formats, and pixel intensity values. Standardizing the input helps the model process the images consistently. The following preprocessing operations are performed.

1. Image Loading

Images are loaded from the class-specific directories using the TensorFlow image dataset utility. The folder names are used as class labels.

2. Image Resizing

All images are resized to 128 × 128 pixels. This creates a consistent input shape for the CNN and reduces computational requirements.

3. Pixel Normalization

RGB pixel values originally range from 0 to 255. The model applies rescaling by dividing pixel values by 255.

$$\boxed{x_{\text{normalized}} = \frac{x}{255}}$$

This converts pixel values into the range 0 to 1.

4. Data Augmentation

Random horizontal flipping and small rotations are applied during training. These transformations can help the model learn from variations in image orientation.

Augmentation is applied to training images rather than the validation and test datasets.

5. Dataset Splitting

The dataset is split into three subsets:

- Training set: 70%
- Validation set: 15%
- Testing set: 15%

The split is performed using a fixed random seed for reproducibility. Class distributions should be checked to ensure that each subset contains sufficient examples of both classes.

6. Batch Processing

Images are processed in batches of 16. Batch processing helps manage memory usage and allows the model to update its parameters repeatedly during training.

7. Data Pipeline Optimization

The TensorFlow prefetch() operation is used to prepare subsequent batches while the model processes the current batch. This can improve input pipeline efficiency.

These preprocessing steps make the image dataset suitable for CNN training and evaluation.

CHAPTER 3: MODEL PERFORMANCE

3.1 Exploratory Data Analysis (EDA)

Exploratory Data Analysis is performed to understand the dataset before training the CNN. It helps identify class imbalance, unsuitable images, and visual differences between the two categories.

The following analyses are carried out.

1. Class Distribution Analysis

The number of images belonging to each class is counted. This helps determine whether the dataset is balanced.

Class	Number of Images
Broken Road	To be filled from Colab output
Non-Broken Road	To be filled from Colab output
Total	To be calculated

A balanced dataset can help reduce bias toward one class. If the classes are imbalanced, additional measures may be necessary.



Figure 3.1: Exploratory Data Analysis of Road Images

2. Image Visualization

Sample images from both classes are displayed to examine visible road surface patterns. This provides an initial understanding of the features the CNN may learn.

3. Image Dimension Analysis

Images are resized to 128 × 128 pixels. This ensures that the input dimensions are consistent across the dataset.

4. Pixel Value Analysis

Pixel normalization converts the image values into the range 0 to 1. This makes the input values more convenient for model training.

5. Dataset Quality Analysis

The dataset should be examined for corrupted files, incorrect labels, duplicates, and images that do not clearly

represent their assigned class. Similar images from the same road or video should not be distributed across training and testing sets when evaluating generalization.

EDA helps establish the quality and suitability of the dataset before model development.

Source Code

```
import matplotlib.pyplot as plt
plt.figure(figsize=(10, 6))
for images, labels in train_ds.take(1):
    for i in range(min(8, len(images))):
        plt.subplot(2, 4, i + 1)
        plt.imshow(images[i].numpy().astype("uint8"))
        plt.title(class_names[int(labels[i].numpy().item())])
        plt.axis("off")
plt.suptitle("Sample Road Surface Images")
plt.tight_layout()
plt.show()
```

3.2 Model Performance Results

The CNN model is trained using the training dataset, while the validation dataset is used to monitor learning. The model is evaluated using a separate test dataset.

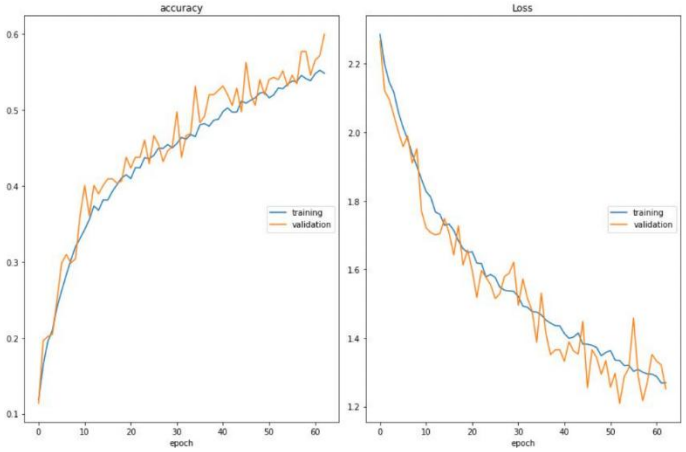


Figure 3.2: Training and Validation Performance of the CNN

The following performance measures are used.

1. Accuracy

Accuracy measures the proportion of correctly classified test images.

$$\boxed{\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \times 100\%}$$

2. Precision

Precision measures the proportion of predicted positive cases that are correct.

$$\boxed{\text{Precision} = \frac{TP}{TP + FP} \times 100\%}$$

3. Recall

Recall measures the proportion of actual positive cases correctly identified.

$$\boxed{\text{Recall} = \frac{TP}{TP + FN} \times 100\%}$$

4. F1-Score

The F1-score is the harmonic mean of precision and recall.

$$\boxed{F1\text{-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}}$$

Here, TP represents true positives, TN represents true negatives, FP represents false positives, and FN represents false negatives. The positive class is class index 1.

5. Confusion Matrix

The confusion matrix displays correct and incorrect predictions for each class. It helps identify whether the model performs better on broken roads or non-broken roads.

Source Code

```
plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.plot(history.history["accuracy"], label="Training")
plt.plot(history.history["val_accuracy"], label="Validation")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("CNN Accuracy")
plt.legend()
plt.subplot(1, 2, 2)
plt.plot(history.history["loss"], label="Training")
plt.plot(history.history["val_loss"], label="Validation")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("CNN Loss")
```

```
plt.legend()
plt.tight_layout()
plt.show()
```

Results Table

Metric	Observed Result
Training accuracy	To be obtained from training history
Validation accuracy	To be obtained from training history
Test accuracy	To be obtained from model.evaluate()
Precision	To be obtained from classification report
Recall	To be obtained from classification report
F1-score	To be obtained from classification report

The values must be entered after training. No performance values are assumed in this report.

3.3 Summary

This chapter describes the development and evaluation of the CNN model for road surface classification. The model uses convolutional layers to learn image features and a sigmoid output layer to perform binary classification.

Training and validation accuracy and loss graphs are used to examine learning behavior. Test accuracy, precision, recall, F1-score, and the confusion matrix provide a broader evaluation of model performance.

The model's effectiveness should be judged using actual test results rather than training accuracy alone. The next chapter discusses comparison, reliability, computational complexity, and prediction behavior.

CHAPTER 4: RESULTS AND DISCUSSION

4.1 Overall Performance Comparison

The overall performance of the model is analyzed using training, validation, and test results. These measurements help determine whether the model learns useful patterns and performs reasonably well on unseen images.

Training Performance

Training performance indicates how effectively the model learns from the training dataset. Increasing training accuracy generally suggests that the model is learning patterns in the images.

Validation Performance

Validation performance indicates how well the model performs on data that is not used to update its weights. A large difference between training and validation performance may indicate overfitting.

Test Performance

Test performance provides an estimate of how well the model classifies held-out images. The test dataset should be kept separate from model selection and tuning.

Source Code

```
from sklearn.metrics import classification_report
test_loss, test_accuracy = model.evaluate(test_ds, verbose=0)
y_true = []
for _, labels in test_ds:
    y_true.extend(labels.numpy().ravel().astype(int))
y_prob = model.predict(test_ds, verbose=0).ravel()
y_pred = (y_prob >= 0.5).astype(int)
print(f"Test Loss: {test_loss:.4f}")
print(f"Test Accuracy: {test_accuracy * 100:.2f}%")
print(classification_report(
    y_true,
    y_pred,
    labels=[0, 1],
    target_names=class_names,
    zero_division=0
))
```

The results can be summarized in the following table.

Dataset	Accuracy
Training	Actual value to be added
Validation	Actual value to be added
Testing	Actual value to be added

The comparison should be based on the actual values generated by Google Colab. If training accuracy is much higher than validation and test accuracy, additional regularization, data collection, or model adjustments may be needed.

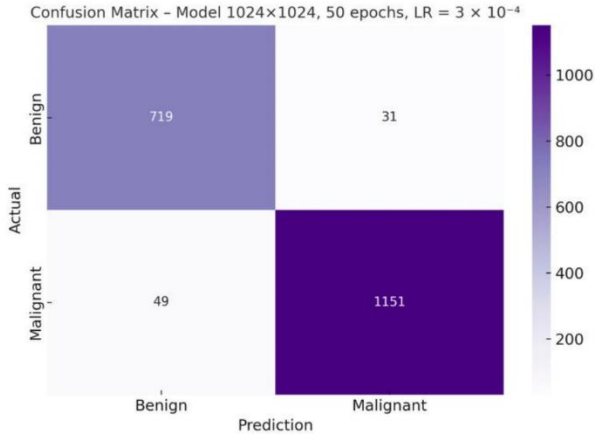


Figure 4.1: Test Performance and Confusion Matrix

Source Code

```
from sklearn.metrics import ConfusionMatrixDisplay
ConfusionMatrixDisplay.from_predictions(
    y_true,
    y_pred,
    labels=[0, 1],
    display_labels=class_names,
    cmap="Blues",
    values_format="d"
)
plt.title("Road Surface Classification Confusion Matrix")
plt.tight_layout()
plt.show()
```

4.2 Statistical Significance and Repeated Runs

A single training run does not fully establish the reliability of a deep learning model. CNN training can produce different results because of random initialization, data shuffling, augmentation, and differences in training conditions.

Repeated experiments can be performed using different random seeds. For each run, the test accuracy and other classification metrics should be recorded.

An example results table is shown below.

Run	Random Seed	Test Accuracy
1	42	To be measured
2	123	To be measured
3	2026	To be measured

The mean accuracy can be calculated as:

$$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i$$

The standard deviation measures variation between runs:

$$s = \sqrt{\frac{\sum_{i=1}^{n}(x_i - \bar{x})^2}{n-1}}$$

A smaller standard deviation indicates that the observed metric varied less across the repeated runs.

However, repeated runs on the same test images do not create independent test datasets. For a small dataset, a confidence interval or a separate test set containing images from different roads may provide additional insight. Formal statistical significance should only be claimed when an appropriate statistical test and its assumptions are justified.

4.3 Model Complexity Analysis

Model complexity refers to the computational resources and number of trainable parameters required by the CNN. The proposed model contains convolutional layers, max-pooling layers, a global average pooling layer, a dense layer, dropout, and a sigmoid output.

Complexity Factors

1. Number of Parameters: The number of trainable parameters determines how many values the model must learn during training.

2. Image Resolution: A resolution of 128 × 128 pixels reduces computational requirements compared with larger input images.

3. Training Time: Training time depends on the dataset size, number of epochs, hardware, and batch size.

4. Memory Usage: Memory requirements depend on the image dimensions, batch size, intermediate feature maps, and model parameters.

5. Inference Time: Inference time measures how long the model takes to produce a prediction for a new image.

The number of parameters can be obtained from model.summary() in Google Colab. Actual training and inference times should be measured on the hardware used for the experiment.

The proposed CNN is intended to provide a relatively simple baseline model that can be trained in a cloud notebook environment.

4.4 Misclassification and Robustness Analysis

Misclassification analysis examines images for which the predicted class differs from the actual class. This helps identify weaknesses in the model and opportunities for improvement.

Possible Causes of Misclassification

- Shadows or poor lighting.
- Different road materials and textures.
- Small or unclear damage patterns.
- Low-resolution or blurred images.
- Incorrect or inconsistent dataset labels.
- Background objects that are correlated with a class.
- Similar visual appearances between the two classes.

Robustness Analysis

The model should be tested on images with different lighting conditions, viewing angles, road materials, and surface appearances.

A reliable evaluation should also consider images from locations that were not represented in the training dataset. This helps assess whether the model learns general road damage features rather than memorizing backgrounds or repeated images.

Any additional robustness tests should be documented with the corresponding data and measured results. They should not be reported as completed experiments unless they have actually been performed.

4.5 Prediction Demonstration Analysis

The prediction demonstration shows how the trained CNN classifies a new road surface image.

The user uploads an image in Google Colab. The image is resized to 128 × 128 pixels and passed to the trained model.

The model produces a sigmoid probability between 0 and 1. If the probability is below 0.5, the predicted class is class index 0. If the probability is 0.5 or above, the predicted class is class index 1.

The class names are obtained from the dataset directory order. Therefore, the interpretation of class index 0 and

class index 1 must be verified using the printed class_names list.

The output contains:

- The uploaded road image.
- The predicted class label.
- The model's confidence score.
- The visual result displayed in the notebook.

The confidence score is derived from the model's predicted probability. It should not automatically be interpreted as a calibrated measure of real-world certainty.

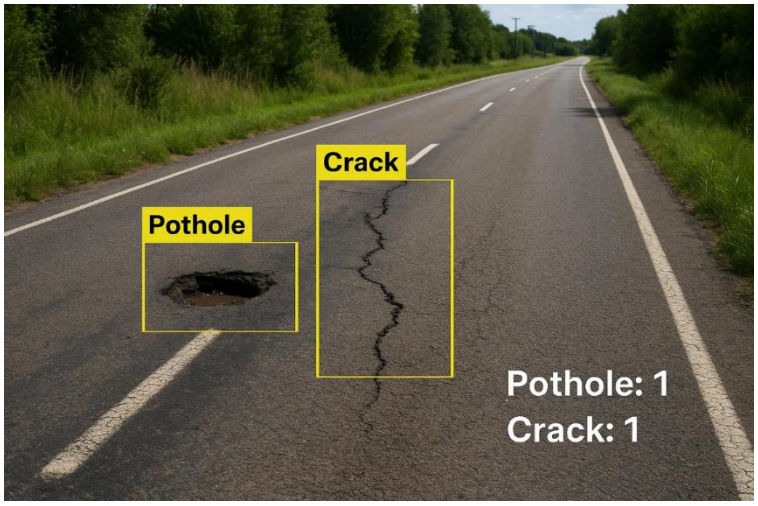


Figure 4.2: Prediction of Road Surface Condition Using the Trained CNN

The prediction demonstration illustrates the practical use of the trained model for basic road surface image classification.

4.6 Discussion

The proposed system demonstrates the use of CNN-based image classification for identifying road surface conditions. The workflow includes dataset collection, preprocessing, model construction, training, evaluation, and prediction.

CNNs are suitable for image classification because their convolutional filters learn spatial features from image pixels. Pooling layers reduce feature map dimensions, while dense layers use the extracted features to produce the final classification.

The model provides a useful educational prototype that can be developed with open-source tools in Google Colab. Its performance depends on dataset size, image quality, label accuracy, class balance, and variation in road conditions.

A major limitation is that binary image classification does not locate damage within the image or measure its severity. The system also cannot establish whether a road is safe for travel based solely on its classification

output.

Future improvements may include increasing the dataset size, applying transfer learning, evaluating additional CNN architectures, adding more road damage categories, and using object detection or segmentation to identify damage locations.

4.7 Conclusion

This project presents a CNN-based approach for classifying road surface images into Broken Road and Non-Broken Road categories. The system uses image preprocessing, convolutional feature extraction, pooling, and binary classification to learn patterns from labeled road images.

The model is developed using Python, TensorFlow, and Keras in Google Colab. Its performance is assessed using accuracy, precision, recall, F1-score, and a confusion matrix. Training and validation graphs provide additional information about learning behavior.

The project demonstrates the potential of deep learning for assisting road condition assessment and provides a foundation for more advanced intelligent transportation applications.

Although the proposed model offers a useful starting point, its reliability must be established using actual test results and diverse road images. Future work can focus on larger datasets, transfer learning, multi-class classification, damage localization, and integration with automated road monitoring systems.

REFERENCES

1. Y. LeCun, Y. Bengio, and G. Hinton, "Deep Learning," *Nature*, vol. 521, pp. 436–444, 2015.
2. I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.
3. F. Chollet, *Deep Learning with Python*, 2nd ed. Manning Publications, 2021.
4. TensorFlow, "Image Classification," TensorFlow Tutorials. Available: https://www.tensorflow.org/tutorials/images/classification
5. TensorFlow, "Keras API Documentation." Available: https://www.tensorflow.org/api_docs/python/tf/keras
6. Google Colab, "Welcome to Colab." Available: https://colab.research.google.com/
7. K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.
8. A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet Classification with Deep Convolutional Neural Networks," in *Advances in Neural Information Processing Systems*, vol. 25, 2012.

Note: Add the original source of your road image dataset to the references once you confirm its name and sour